Tutorial: Mastering Non-Linear Boundaries with SVM Kernels

GitHub Repository: https://github.com/mobasshiremon-uk/SVM-Kernel-Tutorial-UH

License: MIT License

1. Introduction

Support Vector Machines (SVMs) are a powerful class of supervised learning algorithms that have been utilized in this project. The model is in the dataset consisting of features (X) and target label (Y). The feature (X) is the 500 dots mark, and (Y) is the answer "0 for the inner ring and 1 for the outer ring." SVM uses these labels to learn where the boundary between two groups should be. SVM is used here so that we can find the "optimal hyperplane". It is a decision boundary that can separate different classes with the largest possible margin. This margin is the distance between the hyperplane and the closest data points, known as support vectors. The margin is the space or gap between the circular boundaries. By maximizing this space, the model becomes more robust and better at handling new, unseen data.

2. The Problem: Linear Inseparability

In many real-world scenarios, data points cannot be separated by a simple straight line.

- **The Dataset:** For this tutorial, we used Scikit-Learn's `make_circles` function that can generate a synthetic dataset of two concentric circles.
- **The Challenge:** In **Figure 1**, we see the raw synthetic dataset. The red class is centralized and while the blue class forms a surrounding ring. This spatial arrangement creates Linear Inseparability.
- **The Failure of Linear SVM:** Here you can see how the red and blue shaded areas are split by a diagonal line. The blue shaded area contains many red dots. Because a straight line can only split the plane into two halves, it can never "wrap around" the center. This is a classic case of **Underfitting**. The model is too simple to capture the complex circular relationship of the data. A standard linear kernel attempts to draw a straight line through the data. It is mathematically impossible for a straight line to achieve high accuracy, as it will inevitably misclassify nearly half of the points.

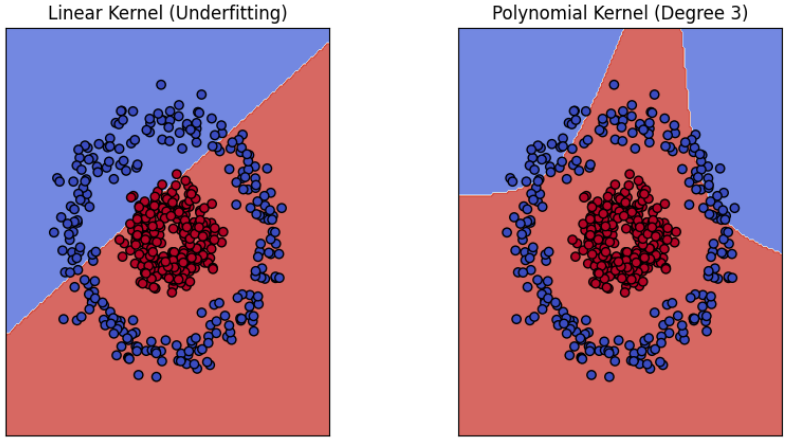

Figure 1: Linear kernel (Underfitting) And Polynormial Kernel

3. The Solution: The Kernel Trick

To solve the problem of linear inseparability, we use the **Kernel Trick** with a **Radial Basis Function (RBF) kernel** as shown in **Figure 3**. The RBF kernel operates as a distance-based classifier. It identifies a central landmark and creates a non-linear, circular decision boundary.

- **The RBF Kernel:** The Radial Basis Function (RBF) is one of the most popular kernels. It measures the distance between points and a central landmark.
- **Results:** If we calculate all the similar points based on their radial distance, the model achieves **100% accuracy**. It perfectly isolates the inner red cluster from the outer blue ring without the need for manual coordinate transformation.

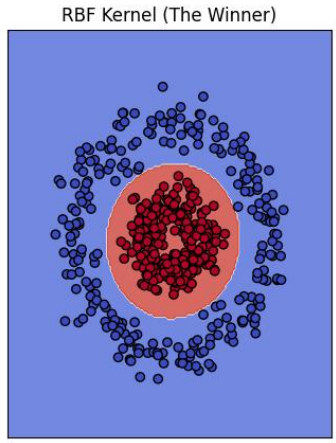

Figure 3: RBF Kernel

4. Visualizing the "Magic Lift" in 3D

How a circle is formed, we must look at the **3D Projection** to understand it. In **Figure 2**, we added a third dimension (the Z-axis) based on the RBF transformation.

- **The Transformation:** In the figure we can see that RBF kernel "lifts" the center red points into a **Mountain**, while the outer blue points remain in a **Valley**.
- **The Hyperplane:** The SVM can easily slide a **flat, 2D sheet** (the 3D Hyperplane) between the mountain and the valley. Because, the data is separated by height.
- **The Projection:** After removing the Z-axis, If we project that flat 3D sheet back down to our 2D map, it appears as the perfect circle we saw in the previous section.

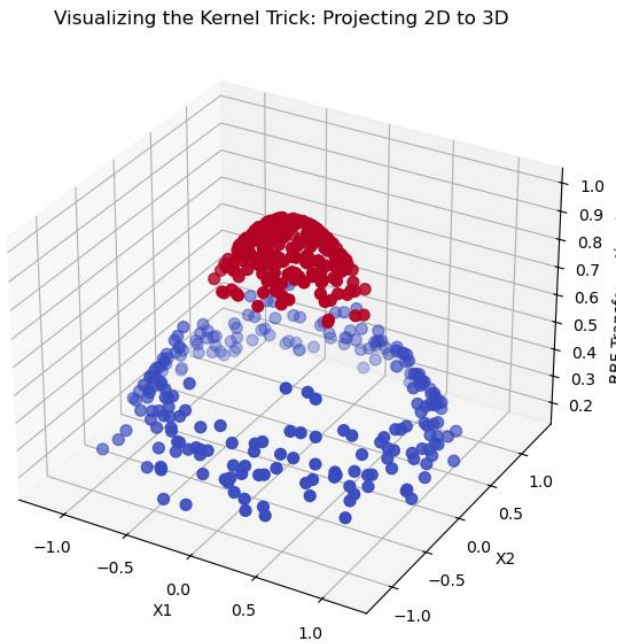

Figure 4:

5. Mastering Hyperparameters: Gamma & C

To truly master the RBF kernel, one must understand the **Gamma** parameter.

- **High Gamma:** In this case he model only see the nearest dots to the boundary. It's has no bid picture. It draws a very wiggly line to make sure it hugs every single dot. This is called **Overfitting.** it's like memorizing the exact position of 500 dots instate of learning that it form a circle.
- **Low Gamma:** The model look at the big picture and it can see the shape of the ring rather that individual dots. Because it can see far away dots, it can draw a smoother more generalized boundary. This works better because even if you add new, slightly messy data late.

- **C Parameter:** This controls the balance between a smooth boundary and perfect classification. A **High C** value acts like a perfectionist. It refused to let even one dot stay on the wrong side of the boundary. It’s forces the model to classify every single training point correctly. Because of that it makes the boundary messy or complex. On the other hand A **Low C** is more relaxed. It’s allows few misclassifications in exchange for a much simpler, smoother boundary.

6. Ethical Considerations in AI

Now machine learning becomes more integrated into modern life. So, we must consider its ethical impact. Like any model SVMs are only as good as the data they are trained on. If an SVM is trained on biased data (e.g., historical data that reflects societal prejudices), the model will "learn" and automate those biases. It is our responsibility to make sure that datasets are representative and fair.

7. Conclusion

After running the code it’s clear that **SVM kernels** and the **RBF kernel** in particular are a game-changer for messy, non-linear data. Before this computer could not separate two circles with just a straight line. After we used the **"Kernel Trick",** it’s possible now to change how we look at the data's geometry.

From this assignment we can see that a model is only as good as its settings. We need to be carefull when tuning **Gamma** and **C Parameter**. It’s help us in finding the right balance between a "wiggly" boundary that overfits and a smooth one that actually works. Whether the data is simple or complex, these tools can give us a reliable way to build highly accurate classifiers for almost any distribution we find in the real world.

7. References:

- **Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992).** *A training algorithm for optimal margin classifiers.*
- **Pedregosa, F., et al. (2011).** *Scikit-learn: Machine learning in Python*
- **Hsu, C. W., et al. (2003).** *A practical guide to support vector classification.*